

Truck Platooning Report

Mohamed Amer,¹ Omar Abdellatif²

Contents

1	Motivation	2
2	System Modelling	2
2.1	System Diagrams	3
2.2	UPPAAL Implementation	4
3	Computer Vision Implementation	5
3.1	Data Collection & initial preprocessing	6
3.2	Pipeline	6
3.3	Computer Vision	7
3.4	Feature Extraction	7
3.5	Support Vector Regression Model	7
4	Results	8
5	Conclusion	8
6	Declaration of Originality	8

Abstract: The increase of vehicles on highways and with the increase of economic needs and logistic supply chains, increased highway accidents. Moreover, the logistic suppliers suffer each year from increasing gas prices and costs of transfer of goods. One of the proposed ideas, is using truck platoons

¹ mohamed-ahmed-mohamed-ali.amer@stud.hshl.de

² Omar.abdellatif@stud.hshl.de

that would be autonomous. Trucks would line up in one row, start moving towards their destination while keeping communication between them so they can detect and predict sudden road conditions. Which in return will increase the road safety [Xi24]. Moreover, Trucks non-aerodynamic shape allow air drag to have a significant effect on their fuel consumption. Moving in a platoon-like formation allow to decrease air drag forces and the overall fuel consumption of the platoon [Mc21]. In this paper we discuss our approach for the overall system, modelling some critical scenarios and using OpenCV and SVR for line detection.

1 Motivation

With rapid economic evolution and the increased logistic demand, freight companies expanded their freight fleets. In return, This increased the possibility for more High way accidents, increased greenhouse gas emission and an increased demand on petroleum. Further studies prove that vehicle emissions are a significant source of atmospheric pollutants. Heavy Duty Vehicles contribute the most regarding NOx emissions. In china, Heavy duty trucks contributed to 80% of the NOx emissions consistently in the past years [Xi24].

More research was conducted on the road safety aspect involving heavy trucks. The Study was conducted in Europe in the year 2015 based on a dataset of accidents. It was found that Heavy trucks were involved in 4.5% of all crashes and 14.2% of fatal crashes, That further indicates that while truck accidents may happen rarely, they contribute more to fatal crashes [Sc21].

Moreover, Statistics showed that most accidents occur because the driver did not have enough time to react to a complicated instantaneous road conditions [Zh21]. This further confirms the need for safe and reliable driving systems with a novel approach such as Truck Platooning. Truck platooning is a convoy of trucks driving together while communicating and coordinating with each other, This represents an approach to improve fuel consumption, lower greenhouse gas emissions, and enhance road safety [Ji24].

This type of system needs to be reliable, safe, and respond quickly to sudden road conditions. In the following report, We tried a different approach of autonomous driving, communication and modelling different types of safety critical scenarios.

2 System Modelling

To begin with, we modelled our system differently, not like the commonly known truck platooning system which basically works with V2V communication. We thought of the whole system: is V2V communication enough for checking the whole platoon status? Is V2V communication enough for the safety of the whole platoon? We asked ourselves these questions, did our research, and found that we can use a centralized communication channel where everything happens from the cloud. All the commands and messages are received and sent via the communication channel, and we have management systems that handle the whole system, as will be shown later in the paper.

2.1 System Diagrams

As for the diagrams, we started with defining our system activity by designing an activity diagram shown in Figure 1. Afterwards, we implemented the whole system in the sequence diagram shown in Figure 2 so we could plot our time behaviour and model the time taken by the system. In the sequence diagram, we can observe that the communication is happening via the communication channel. When the platoon is roaming the streets, the command starts from the leader truck; it doesn't go directly to the following trucks as in traditional V2V. Instead, it goes from the leader truck to the communication channel and therefore to the management systems within the system. As we can see, we have the platoon manager that manages the whole platoon. As stated above, we went with the centralized approach in order to check the state of the whole platoon. Thus, all the communication is happening via the cloud management system.

In the case of a truck wanting to couple to our platoon, it starts by sending a request to the communication channel indicating it wants to couple, and all the logical functions are handled within the cloud, back to the truck telling it either to couple or abort coupling. Afterwards, we wanted to simulate our sequence diagram in order to observe the system behaviour with respect to time, so we started with the UPPAAL implementation so we could set timed automata and simulate the system.

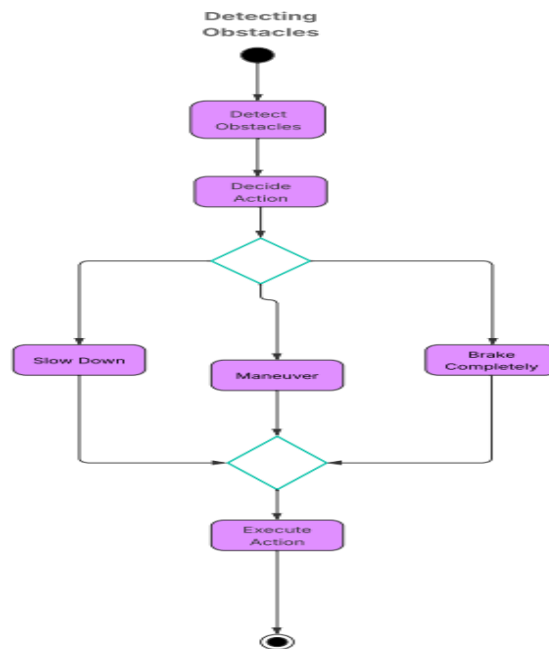


Fig. 1: Activity diagram for the Maneuvering scenario.

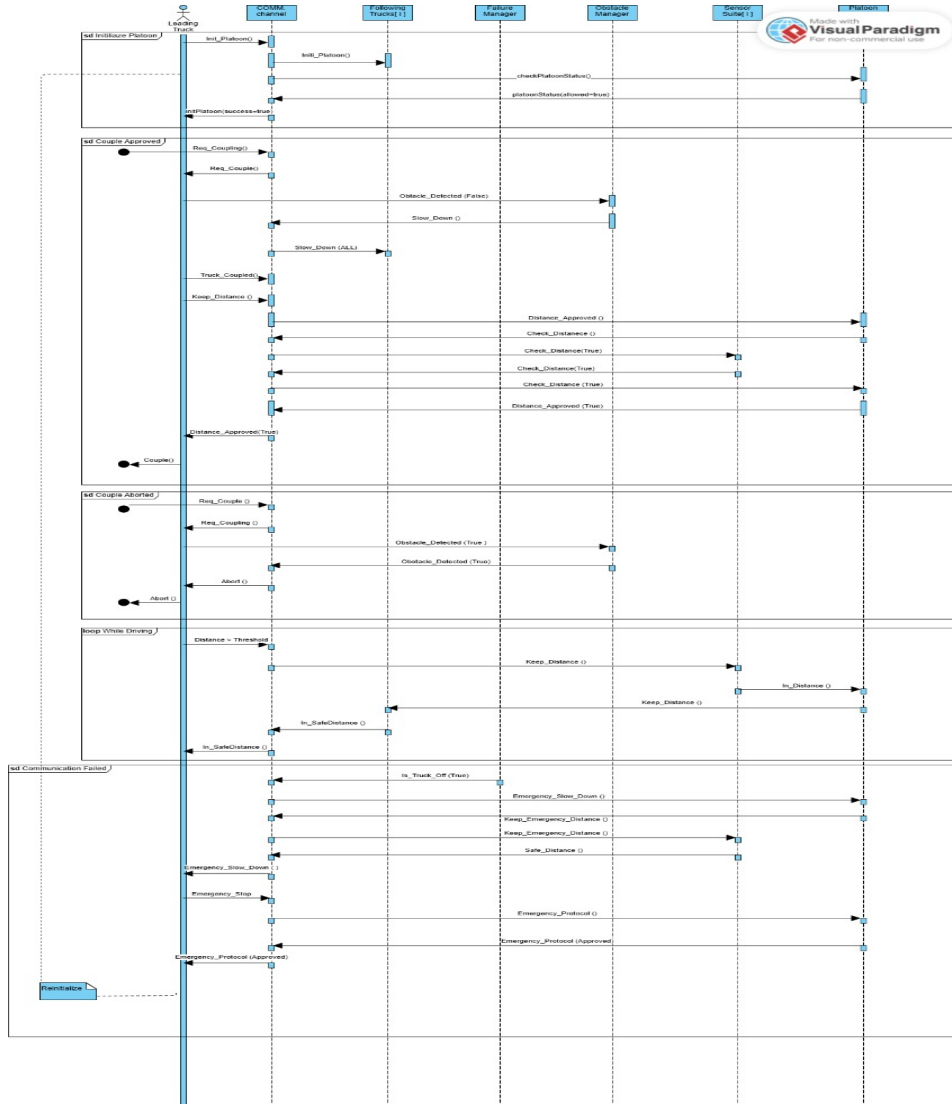


Fig. 2: Sequence Diagram of the entire selected system scenarios

2.2 UPPAAL Implementation

In our UPPAAL implementation, we defined our templates with respect to the sequence diagram, as these templates mirror the sequence diagram columns. We then started by defining our channels that the templates will use to communicate, as we declared them in the

global declarations so they would be visible to all templates. Afterwards, we started defining our states within each template. We declared our clock variables; some of them were added as global variables and some of them were added within the declaration of the templates themselves. The clock variables were used as invariants to define how much time we will stay in a state before transitioning, or as guards to control how long we will be transitioning from one state to another. Unfortunately, we faced deadlocks in our system, and we did our best to solve this issue. Eventually, we modelled our system, designed it, understood our states and time behaviour, and now it is time to start implementing our system. In the following Figure 3, the full integration of our 2 modelled scenarios in UPPAAL.

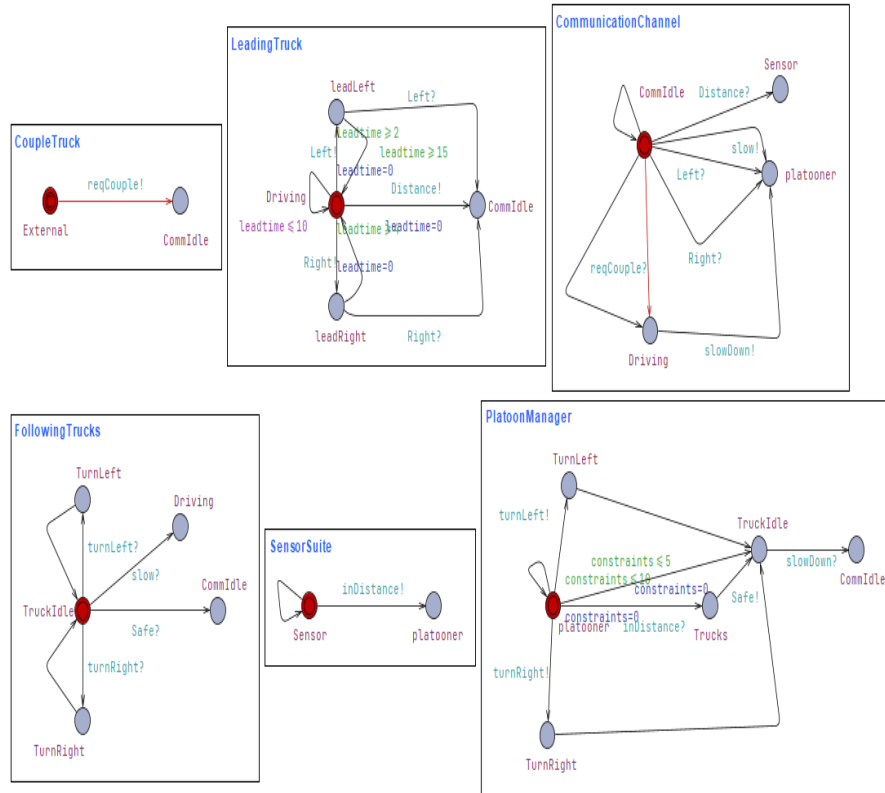


Fig. 3: Full integration of our 2 modelled scenarios in UPPAAL

3 Computer Vision Implementation

Our approach was to use OpenCV along with a lightweight machine learning model that would not need much processing power and be having a really quick response to further

more address that autonomous driving systems are safety critical and the algorithm needs to have a near real-time response. While the use of deep learning and neural networks may have slightly better accuracy, though the computational cost of that would be more than the benefit acquired [Zh21].

3.1 Data Collection & initial preprocessing

The data used was collected in a CARLA environment. The truck was made to go around a small city with no traffic for about 1 hour. During driving around, the truck was made to go through all possible scenarios. The truck was driven around with different speeds, different positions relative to the line, and going into both possible driving directions performing right and left turns. The data collected was images with a frequency of 1 image per second. The dataset we collected was about 8Gb in size initially. The data was injected with some noisy pictures and pictures from different views of the truck. This data was found to be having a relatively smaller size than the data needed for the computer vision algorithm. Those noise images were then moved into a different folder to avoid exhausting the algorithm with out of context pictures. After removing the noise, The dataset ended up being around 1000 images.

3.2 Pipeline

According to several techniques to process images in several researches, a merging of many techniques pipeline was proposed. Fig. 1 illustrates the proposed image processing, feature extraction, and machine learning pipeline [Zh21, MT23, YZ15].

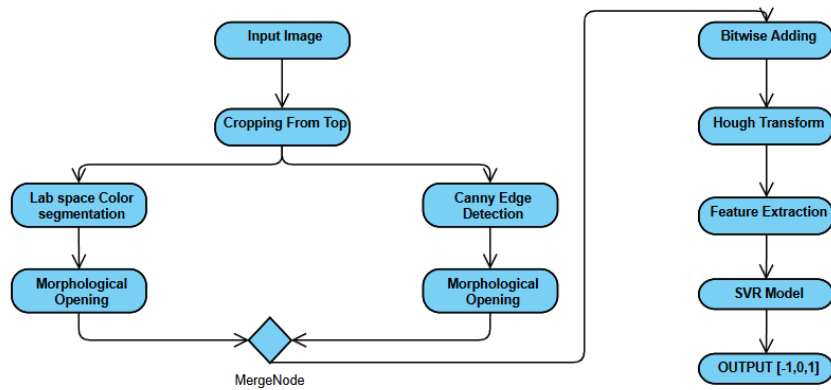


Fig. 4: Pipeline of implementation

At first, A Graphical tuning interface was made to tune the parameters for each process in the OpenCV pipeline.

3.3 Computer Vision

According to previous research, The first step of input image processing cropping the image from the top to remove any unnecessary data from the environment [Zh21]. Then as shown in fig. 1, The image undergoes 2 different processing pipelines. The reason for that split is that the environment used to collect the data and test it was full of extreme light variations, objects having nearly the same colour as the line we wanted to follow. This approach was meant to be testing if we can nearly eliminate all the noise in the data as there were many noise in the data collected. Using Canny edge and colour segmentation is a well known technique image processing for line or lane detection that is used frequently in research projects [Zh21, MT23]. To further eliminate the noise after having those 2 separate masks, a bit-wise addition of the 2 processed images was taken as the final mask then undergoing Hough transform for line detection. As mentioned in [YZ15], Hough Transform is relatively computationally expensive. Though the difference when using other techniques to detect the line, Hough transform was the best performing.

3.4 Feature Extraction

After undergoing the entire OpenCV pipeline, the processing done was used to extract 19 of the main features that defines the detected line. Then all the features were dumped into a csv file to be used for training the model. Some of the most important features are shown in Table. 1.

Features	Description
Centroid X-coordinate	Represents the horizontal position of the centroid of the detected line in the image.
Average X of Line Pixels	Represents the average X-coordinate of all the pixels that make up a detected line in the image.
Number of Detected Lines	Number of lines detected in the input image.
Normalized Mask Centroid X	Represents the X-coordinate of the centroid of the detected line in the image, normalized using the size of the image.

Tab. 1: Main features extracted

3.5 Support Vector Regression Model

The model used to detect the line position from the extracted features was the support vector regression model. The support vector regression algorithm is based on the support vector

machines. It works by finding a function that approximates the input-to-output relationship with a minimal error value not exceeded [SS04]. After tuning the hyper-parameters via a grid search, a linear kernel was found to be the best performing. The model reached a validation accuracy of approximately 97 percent. The model and the scalar features were saved as joblib file for the integration with the CARLA environment.

4 Results

By loading the model and testing it into the CARLA environment, our model performed fairly good at detecting the line. The truck was able to follow the line correctly with some issues when noise or different lighting scenarios were introduced. Moreover, when checking the CPU load for our model it was minimal, nearly consuming about 5 percent of the capability of the CPU which was satisfactory.

5 Conclusion

In conclusion, our openCV pipeline was not the ideal model for this task. Though many iterations were made to fix the noise problems in the virtual environment, some still persisted. This further confirms that there is a need of more research to develop a robust and a noise proof image processing pipeline that would eliminate the noise effects on the model. Furthermore, the model performed really well in terms of accuracy, and the openCV integration with the SVR model provided a computationally light weight model that has a really low run-time and can be further reused to produce a low latency system.

6 Declaration of Originality

I, Mohamed Amer and Omar Abdellatif, herewith declare that I have composed the present paper and work by myself and without the use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated by full details of the publications concerned. The paper and work in the same or similar form have not been submitted to any examination body and have not been published. This paper was not yet, even in part, used in another examination or as a course performance. I agree that my work may be checked by a plagiarism checker.

Bibliography

- [Ji24] Jiang, Luo; Kheyrollah, Javad; Koch, Charles; Shahbakhti, Mahdi: Cooperative truck platooning trial on Canadian public highway under commercial operation in winter driving conditions. *Proceedings of the Institution of Mechanical Engineers Part D Journal of Automobile Engineering*, 04 2024.
- [Mc21] Mcauliffe, Brian; Smith, Patrick; Raeesi, Arash; Hoffman, Mark; Bevely, David: Track-Based Aerodynamic Testing of a Two-Truck Platoon. volume 3, 04 2021.
- [MT23] Mulgund, Akshay; T, Shivakumara: Enhanced Lane Detection Techniques on Structured Roads using OpenCV. *International Journal of Scientific Research in Engineering and Management (IJSREM)*, 7(8):1–6, 2023. SJIF Rating: 8.176.
- [Sc21] Schindler, Ron; Jansch, Michael; Johannsen, Heiko; B'alint, Andr'as: An analysis of European crash data and scenario specification for heavy truck safety system development within the AEROFLEX project. 2021.
- [SS04] Smola, Alex; Schölkopf, Bernhard: A tutorial on support vector regression. *Statistics and Computing*, 14:199–222, 08 2004.
- [Xi24] Xiao, Tong; Qin, Zhengtao; Lu, Yougeng; Chao, Yuan; Yang, Chao; Yuan, Quan: Reexamining exposure from truck emissions considering daily movement of individuals. *Transportation Research Part D: Transport and Environment*, 136:104441, 2024.
- [YZ15] Yang, Xu; Zhang, Ling: Research on Lane Detection Technology Based on OpenCV. In: *Proceedings of the 3rd International Conference on Mechanical Engineering and Intelligent Systems (ICMEIS 2015)*. Atlantis Press, Chongqing, China, 2015.
- [Zh21] Zhu, H.G.: An Efficient Lane Line Detection Method Based on Computer Vision. *Journal of Physics: Conference Series*, 1802:032006, 2021.